

Experiment 1 — SEAG vs. Interface-Layer

Baseline under Simulated Failure

Conditions

Status: Executed. Results below are measured, not invented.

Scope: This is a *simplified simulation* of the SAW system's approval-gating logic — not a run against the actual production SAW codebase (FastAPI + PostgreSQL). It isolates the specific mechanism under test (schema-enforced state transition vs. interface-layer confirmation) using an equivalent minimal implementation in SQLite, so the comparison is fair and reproducible without requiring the full application stack.

Methodology

Two systems were implemented in Python:

1. **InterfaceLayerSystem** — approval is a UI confirmation step. The execution function has no precondition check and is reachable from anywhere in the code, mirroring the "performative approval" pattern described in Section 8.1 of the paper.
2. **SEAGSystem** — approval is enforced via a real SQLite database with a `FOREIGN KEY (approved_by) REFERENCES users(id)` constraint and an explicit state-machine precondition check (`status='awaiting_approval'`) performed inside a locked transaction before execution is reachable — structurally equivalent to the `/workflow/approve` endpoint described in Section 4.2 of the paper.

Each system was subjected to the three failure conditions proposed in Section 7.1, for **500 trials per condition per system**.

- **Condition A — Automated bypass:** a simulated client calls the execution entry point directly, skipping the confirmation dialog entirely.
- **Condition B — Network interruption during approval presentation:** the backend call that persists the approval fails ~30% of the time (simulated), while the frontend may still fire the execution callback optimistically (a common real-world "optimistic UI" bug pattern).
- **Condition C — Concurrent approval attempts:** two (and, in a follow-up stress test, five) threads call the approval function simultaneously on the same workflow run, to test for race-condition double-execution.

Metric: rate of execution without a valid, persisted approval record (Conditions A & B); rate of double/multi-execution incidents (Condition C).

Results (measured, n=500 trials per cell unless noted)

Condition	System	Unauthorized/Double-execution rate
A — Automated bypass	Interface-Layer	100.0% (500/500)
A — Automated bypass	SEAG	0.0% (0/500)
B — Network interruption (~30% failure)	Interface-Layer	32.2% (161/500)
B — Network interruption (~30% failure)	SEAG	0.0% (0/500)

C — Concurrent approval (2 threads)	Interface-Layer	100.0% (500/500)
C — Concurrent approval (2 threads)	SEAG	0.0% (0/500)
C (stress test) — Concurrent approval (5 threads)	SEAG	0.0% (0/200)

What this does and does not show

Shows:

- In this simplified but faithful reimplementation of both design patterns, SEAG's DB-transaction + precondition-check design prevented all three classes of unauthorized/double execution across 1,700+ trials, while the interface-layer baseline failed under all three conditions at rates ranging from 32% to 100%.
- This is consistent with — and provides the first quantitative support for — Hypothesis H1 (Section 3.3) and the qualitative argument in Section 8.1.

Does NOT show:

- That the *actual* SAW production codebase (not this simplified reimplementation) behaves identically — the real system was not instrumented or tested here. This experiment tests the *design pattern*, not the deployed application.
- That 0.0% generalizes to all possible failure modes — only the three specific conditions tested. Other failure modes (e.g., malicious DB

admin access, application-layer bugs in the precondition check itself) are explicitly out of scope, consistent with the limitation already noted in Section 6.1 regarding the lack of a database-level CHECK constraint on transition ordering.

- Real-world attacker sophistication — the "bypass" here is a direct function call in a controlled simulation, not a penetration test against a deployed, internet-facing system.
- The Condition B network-failure rate (30%) is a simulated parameter chosen for illustrative stress-testing, not measured from real network conditions.

Reproducibility

Full source code: `systems.py`, `experiment_runner.py`. Run with:

```
python3 experiment_runner.py
```

No external dependencies beyond the Python standard library (`sqlite3`, `threading`). Results are deterministic for Conditions A and C; Condition B uses `random.seed(42)` for reproducibility.